university of groningen

faculty of science and engineering

Verification of Security and Privacy concepts in BPMN Choreography diagrams

Bachelor's Project Computing Science

July 2026

Author: Vitalii Sikorski
Student Number: S5622360
First supervisor: Heerko Groefsema
Second supervisor: Alexander Lazovik

Contents

Abstract

Business Process Model and Notation (BPMN), standardized by the Object Management Group (OMG) [15], is a graphical language for modeling business processes and interactions. Choreography diagrams in BPMN are specifically designed to describe interactions between independent participants without exposing their internal processes. However, their formal semantics mainly capture control flow and do not explicitly represent privacy-related constraints. Privacy-Enhanced BPMN (PE-BPMN) [16] allows the annotation of Separation-of-Duties (SoD), Binding-of-Duties (BoD), and Division-of-Knowledge (DoK) [9], but standard Petri net–based verification tools cannot automatically check whether these constraints hold during execution. This thesis addresses this gap by extending the BPMVerification tool [8] with security-aware semantics for BPMN choreography diagrams. Choreography tasks are enriched with annotations describing required and produced knowledge, as well as duties created as a consequence of execution. The Petri net transformation is extended to track knowledge and duty states for each participant, modeling participants as roles with dynamic knowledge and duty sets. SoD ensures no participant holds conflicting responsibilities, BoD guarantees certain tasks are executed by the same participant, and DoK prevents a participant from holding all knowledge items in a predefined critical set. Privacy constraints are expressed as reachability conditions and verified using model checking. The extension augments both the Petri net transformation and the specification generation process, enabling automated verification alongside behavioral checks. The contributions include: (1) a formal model for tracking knowledge and duty states in BPMN choreographies, (2) an extension to the BPMVerification tool with privacy-aware semantics, (3) a privacy specification language for SoD, BoD, and DoK constraints, and (4) an evaluation demonstrating feasibility on realistic process models. The results show that privacy constraints can be effectively verified alongside behavioral properties, providing a practical solution for ensuring privacy compliance in distributed business processes.

1 Introduction

Business Process Model and Notation (BPMN) [15] is a widely adopted industry standard for modeling business processes. Its intuitive, flowchart-like notation makes it accessible to both technical and non-technical stakeholders, enabling effective communication between business analysts and software engineers [6]. BPMN choreography diagrams, in particular, are designed to describe message exchanges between independent participants while abstracting from their internal behavior. Because of their clear structure and standardized notation, they are frequently chosen for modeling distributed collaborations in domains such as supply chain management, healthcare, and financial services [19].

The popularity of BPMN choreographies stems from their ability to capture cross organizational interactions without exposing the internal processes of each participant. This abstraction is both a strength and a limitation. While it enables high-level modeling of collaborations, it also obscures important security and privacy properties that may depend on the internal knowledge and duty states of participants. A choreography may satisfy all control-flow correctness checks while still allowing undesirable accumulations of knowledge or conflicting responsibilities along certain execution paths.

For formal analysis, BPMN models can be translated into Petri nets [13], a mathematical modeling language that provides a rigorous foundation for analyzing system behavior. This transformation enables the application of established model-checking techniques to verify reachability properties [5]. The BPMVerification tool [8], developed by Groefsema et al., implements this approach by transforming BPMN diagrams into Petri nets and checking for control-flow properties such as deadlocks and livelocks. This approach works effectively for behavioral verification, but privacy and security requirements are not reflected in the underlying execution semantics.

Privacy concerns naturally arise in choreography settings. When multiple organizations collaborate, sensitive information is exchanged, and responsibilities are distributed among participants. Ensuring that no participant acquires excessive knowledge or conflicting duties becomes a critical requirement. Privacy-Enhanced BPMN (PE-BPMN) [16] allows the annotation of properties such as Separation-of-Duties (SoD), Binding-of-Duties (BoD), and Division-of-Knowledge (DoK) [9]. However, even when such constraints are added as annotations, they remain external to the formal model because the Petri net transformation of the BPMVerification package does not track knowledge or duty states. Consequently, a choreography may satisfy all behavioral checks while still allowing privacy violations along certain execution paths.

This thesis addresses this gap by extending the BPMVerification tool with security-aware semantics for BPMN choreography diagrams. The central research question is: *How can privacy constraints such as SoD, DoK, and BoD be incorporated into a formal execution semantics for BPMN choreography diagrams to enable automated verification?* This raises several sub-questions:

- Which privacy and security conditions can be defined for BPMN choreographies?
- How can SoD, DoK, and BoD conditions be operationally represented using Petri nets?
- How can SoD, DoK, and BoD verification be automated alongside standard behavioral checks?

To address these questions, this thesis extends the BPMVerification tool by defining execution-level semantics that tracks participant knowledge and duty states. Choreography tasks are annotated with knowledge requirements, produced knowledge, and assigned or bound duties. The Petri net transformation is modified to include corresponding places for knowledge and duties, enabling privacy constraints to be formulated as reachability properties and verified automatically. The approach introduces three privacy constraint types:

1. **Separation of Duties (SoD)** prevents a participant from being assigned a critical set of responsibilities. Verification checks for reachable markings where a participant holds mutually exclusive duty tokens.
2. **Binding of Duties (BoD)** requires that specific duties be assigned to the same participant. Unlike SoD, which prevents conflicts, BoD enforces grouping of responsibilities.
3. **Division of Knowledge (DoK)** prevents a participant from simultaneously holding all knowledge items in a predefined critical set, protecting sensitive information from being fully accessible by a single actor.

The proposed framework extends the BPMVerification package by augmenting the Petri net with additional places and transitions that track duty and knowledge states. The specification generation is also enhanced to include privacy formulas that are checked against the augmented net. This is achieved through a modular architecture where privacy modules can be dynamically loaded based on the constraints specified in a privacy configuration file. The augmentation operates on the existing Petri net structure, adding:

- **Duty places** for each participant-duty combination, which accumulate tokens when a participant executes tasks that create duties.
- **Knowledge places** for each participant-knowledge combination, which track knowledge acquisition and consumption.
- **Violation detectors** that fire when a participant accumulates conflicting duties, a critical knowledge set, or when required duties are bound.

The verification pipeline follows a clear sequence: BPMN choreography diagrams are transformed into Petri nets, which are then augmented with privacy-aware places and transitions. The privacy constraints are expressed as CTL formulas and checked using the NuSMV model checker [14]. If a violation is reachable, the model checker produces a counterexample trace showing the execution path that leads to the violation, which is invaluable for diagnosing root causes. Thus, the main contributions of this thesis are:

1. A formal model for tracking participant knowledge and duty states in BPMN choreography diagrams, extending the existing Petri net transformation.
2. An extension to the BPMVerification tool that automatically augments Petri nets with privacy-aware semantics, supporting SoD, DoK, and BoD constraints.
3. A privacy specification language for expressing privacy constraints in XML, which is parsed and transformed into verification artifacts.
4. An evaluation demonstrating the feasibility of the approach on realistic process models, including both reachable and unreachable violation scenarios.

The remainder of this thesis is structured as follows. Section 2 discusses related work in BPMN verification and privacy-aware process modeling. Section 3 provides background on BPMN choreography diagrams, Petri nets, privacy constraints, and model checking. Section 4 presents the requirements for the proposed framework. Section 5 describes the method, including the formal model and the verification approach. Section 6 details the implementation, covering the architecture, privacy modules, and augmentation process. Section 7 evaluates the framework on a set of test cases. Finally, Section 8 concludes the thesis and outlines directions for future work.

2 Related Work

This section reviews the existing literature on BPMN verification and privacy-aware process modeling, identifying the gap that this thesis addresses. The discussion is structured around three main areas: BPMN verification tools, privacy-enhanced BPMN extensions, and formal approaches to privacy verification in business processes.

2.1 BPMN Verification Tools

Several tools have been developed to enable formal verification of BPMN models. These tools typically transform BPMN diagrams into formal models such as Petri nets, which can then be analyzed using model checking techniques. The BPMVerification package [8], developed by Groefsema et al., provides a framework for analyzing Petri net-based business process models. The tool supports the verification of control-flow properties such as reachability, deadlock freedom, and compliance with process constraints. It is built on top of BPMPetriNet, a library that handles the underlying Petri net representation and manipulation.

The approach taken by BPMVerification is representative of a broader class of tools that leverage Petri nets for BPMN analysis. The transformation from BPMN to Petri nets preserves control-flow semantics while abstracting away data and resource aspects. This abstraction is suitable for verifying behavioral properties but limits the ability to reason about security and privacy concerns that depend on the internal state of participants. While the original work by Groefsema focused on control-flow correctness, later extensions have explored compliance checking for various types of process constraints.

Other notable tools in this space include the BPMN-Q query language [2], which allows querying BPMN models for compliance with patterns, and the LoLA model checker [18], which is often used for Petri net analysis. However, these tools primarily focus on control-flow and data-flow verification, with limited support for security or privacy properties. None of the existing tools provide native support for tracking participant knowledge states or duty assignments necessary to verify SoD and DoK constraints in choreography diagrams.

The BPMVerification package serves as the foundation for this thesis. The choice is motivated by its mature implementation, active maintenance, and compatibility with the broader BPMN ecosystem. By extending BPMVerification rather than developing a new tool from scratch, this thesis leverages existing infrastructure for net transformation, specification parsing, and model checking integration.

2.2 Privacy-Enhanced BPMN (PE-BPMN)

Privacy-Enhanced BPMN (PE-BPMN), introduced by Pullonen et al. [16], extends standard BPMN with privacy-related annotations. PE-BPMN allows modelers to specify constraints related to data visibility, access control, and information distribution directly within the BPMN model. The notation includes annotations for designating which participants are

permitted to observe or process certain data items, as well as constraints such as SoD and DoK.

The formal semantics of PE-BPMN are defined using a process-calculus-based notation, providing a rigorous foundation for reasoning about privacy properties. This formalization enables the verification of privacy constraints, at least in principle. However, the formal semantics are not directly supported by standard Petri net-based verification tools. Consequently, while PE-BPMN enhances the expressive power of BPMN for privacy-aware modeling, it does not provide an automated verification pipeline that can be practically applied to real-world models.

Several extensions of PE-BPMN have been proposed to address this limitation. Some approaches translate PE-BPMN annotations to other formal languages, such as Answer Set Programming or SAT [4]. While these approaches demonstrate the feasibility of reasoning about privacy constraints, they often require specialized solvers that are not integrated into existing BPMN verification pipelines. The separation between modeling and verification remains a significant challenge.

The work by Groll et al. [9] provides a comprehensive survey of SoD as a security principle, covering its formal definitions, enforcement mechanisms, and application domains. The survey highlights the importance of systematic verification of SoD constraints in critical systems, emphasizing the need for practical tools that can automate such checks. However, the survey does not address the specific challenges of modeling and verifying SoD in BPMN choreography diagrams.

2.3 Formal Verification of Privacy in Business Processes

Several approaches have explored the formal verification of privacy properties in business processes. Belluccini et al. [3] propose a model-based verification framework for data protection mechanisms in collaborative business processes. Their work focuses on collaborative BPM models rather than choreography diagrams and relies on a process-algebraic framework instead of Petri nets. The approach uses the SMT-LIB language to encode privacy constraints and employs SMT solvers for verification. While the approach is powerful, it is not integrated into existing BPMN verification tools and requires a significant shift from the standard Petri net-based analysis paradigm.

The use of Petri nets for modeling and verifying security properties has been explored in various contexts. Knorr [11] investigates the use of Petri nets for modeling access control and separation of duty constraints. The work demonstrates how Petri nets can be extended with additional places and transitions to model security-relevant aspects of a system. However, the approach is not specifically tailored to BPMN choreographies and does not address the particular challenges of choreography diagrams, such as message-based interactions and multi-participant coordination.

Other relevant work includes the verification of workflow security constraints by Atluri et al. [1] and the use of colored Petri nets for modeling and analyzing security properties [10]. These approaches demonstrate the general applicability of Petri nets for security verification but are not integrated into a BPMN-specific verification toolchain. The gap between security modeling and practical verification remains open.

Recent work by Montali et al. [12] explores compliance checking of business processes using finite-state automata and temporal logic. Their approach focuses on control-flow compliance but acknowledges the need to extend compliance checking to include data and resource aspects, including security and privacy constraints. The authors identify the tracking of participant knowledge and duty states as an important direction for future research.

2.4 Gap Analysis

The analysis of existing approaches reveals a clear gap between privacy-aware modeling and practical verification in BPMN choreography contexts. While BPMN provides expressive modeling capabilities, its informal semantics prevent direct formal verification. PE-BPMN enhances the specification of privacy constraints but does not provide an automated verification pipeline. Formal verification approaches such as those by Belluccini et al. provide rigorous semantics but rely on different formal models (process algebra, SMT-LIB) that are not integrated into the existing BPMN verification infrastructure.

This thesis addresses this gap by building directly on the BPMVerification framework, which provides the core infrastructure for BPMN-to-Petri-net transformation and model checking. By extending this existing framework, the proposed approach maintains compatibility with the broader BPMN ecosystem while adding privacy-aware semantics. The use of Petri nets as the underlying formal model ensures that privacy constraints can be verified using the same reachability-based verification techniques that are already employed for behavioral analysis.

The choice to extend BPMVerification, rather than developing a new tool from scratch, is justified by the availability of mature infrastructure for net manipulation, specification parsing, and model checking integration. This approach allows the thesis to focus on the novel aspects of privacy tracking and verification while reusing proven components for the foundational aspects. The resulting framework bridges the gap between privacy-aware modeling and automated verification, enabling practical analysis of SoD, DoK, and BoD constraints in BPMN choreography diagrams.

3 Background

3.1 BPMN Choreography Diagrams

BPMN Choreography Diagrams Business Process Model and Notation (BPMN) is a graphical notation standardized by the Object Management Group (OMG) for modeling business processes [15]. Its intuitive flowchart-like notation makes it accessible to both technical and non-technical stakeholders, allowing business analysts to convey process logic to software engineers effectively [6]. BPMN defines three primary diagram types: Process diagrams (internal workflows), Collaboration diagrams (interactions between participants with message flows), and Choreography diagrams (global message exchanges abstracting internal behavior) [19]. Choreography diagrams are particularly relevant for cross-organizational collaborations, where privacy concerns naturally arise as sensitive information and responsibilities are distributed among independent participants. Figure 1 illustrates a BPMN choreography diagram showing message exchanges between a Patient and a Doctor. Each task represents a message exchange, with the initiating participant and receiving participant clearly identified. The diagram abstracts away internal process details, focusing solely on the interaction pattern between participants.

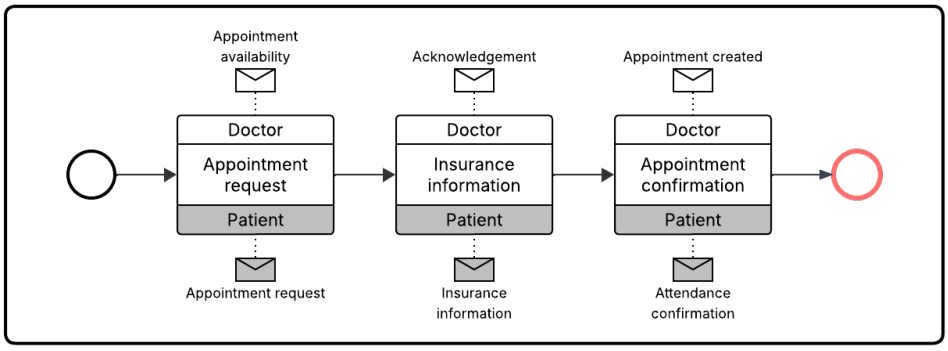

Figure 1: Example of a BPMN Choreography Diagram.

However, BPMN's informal semantics lack built-in mechanisms for representing or verifying privacy constraints such as Separation of Duties or Division of Knowledge [20]. While annotations can be added using extensions like Privacy-Enhanced BPMN (PE-BPMN) [16], these remain external to the formal model unless explicitly incorporated into the execution semantics. This limitation motivates the need for formal verification approaches that can reason about both behavioral correctness and privacy compliance within the same framework.

3.2 Petri Nets

Petri nets provide a formal foundation for analyzing BPMN models through transformation. A Petri net is a bipartite directed graph consisting of four fundamental elements [13]:

1. **Places** (depicted as circles): Represent conditions or states that can hold tokens.

2. **Transitions** (depicted as rectangles): Represent events or actions.
3. **Arcs** (depicted as arrows): Connect places to transitions and vice versa, defining token flow.
4. **Tokens** (depicted as dots): Represent the presence of a condition or availability of a resource.

A **marking** is a distribution of tokens across all places, capturing the current state. The execution is governed by the firing rule: a transition is **enabled** if all its input places contain at least one token. When an enabled transition fires, it consumes one token from each input place and produces one token to each output place, resulting in a new marking. An example can be observed in Figure 2.

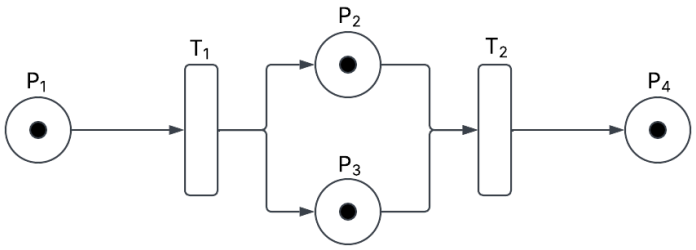

Figure 2: Example of a Petri Net marking.

The **reachability problem** asks whether a particular marking can be reached from the initial marking through a sequence of firings [17]. This problem is fundamental to verification, as many correctness properties can be expressed as reachability conditions. For example, a violation state where a participant holds conflicting duties can be represented as a marking, and the verification task becomes checking whether that marking is reachable. The formal nature of Petri nets enables the application of model checking techniques to BPMN models.

3.3 Privacy Constraints: SoD, DoK, and BoD

Privacy in collaborative processes is commonly expressed through three constraint types that impose restrictions on how knowledge and duties are distributed among participants.

Separation of Duties (SoD) ensures that no single participant is assigned conflicting responsibilities, preventing fraud, errors, or excessive authority [9, 16]. For example, the same employee should not be permitted to both create a purchase order and approve payment for that order. In formal terms, SoD violations occur when a participant holds a set of mutually conflicting duties. Verification checks for reachable markings where duty places for the same participant contain conflicting duty tokens.

Division of Knowledge (DoK) prevents participants from possessing all elements of a critical knowledge set, protecting sensitive information from being fully accessible by a single actor [16]. For instance, an employee should not simultaneously know both the contract

details and the payment information. DoK violations occur when a participant accumulates all knowledge items in a predefined critical set. In the Petri net model, this is detected by checking whether knowledge places for the same participant contain all critical knowledge tokens.

Binding of Duties (BoD) complements SoD by requiring that certain tasks be performed by the same participant, ensuring consistency and accountability [9]. While SoD prevents conflicts, BoD enforces that specific duties are grouped together. For example, the same person who approves an order should also approve the corresponding payment.

These constraints can be specified as annotations on choreography tasks, but they remain external to formal models unless explicitly incorporated into execution semantics [7]. This thesis addresses this by extending the Petri net transformation to track knowledge and duty states, enabling automated verification of all three constraint types.

3.4 Verification Pipeline

The verification pipeline transforms the Petri net into formal structures suitable for model checking. The Kripke structure abstracts the state space, where states correspond to markings and transitions represent firings [5]. Atomic Propositions (APs) are Boolean conditions over states, mapping each state to the set of propositions that hold. In the BPMVerification tool, APs correspond to enabled transitions, representing transitions that are enabled and can fire from a given marking. For example, an AP may indicate that a specific transition is enabled in a particular state.

Specifications are expressed in Computation Tree Logic (CTL), a branching-time temporal logic used for model checking [5]. CTL formulas combine path quantifiers with temporal operators. The path quantifiers are **A** (for all paths) and **E** (for some path), and the temporal operators are **F** (finally), **G** (globally), **X** (next), and **U** (until). Key CTL formulas include:

- **AG p** (All paths Globally p): p holds in every state on all paths.
- **EF p** (Exists path Finally p): p holds in some state on some path.
- **AX p** (All paths neXt p): p holds in the next state on all paths.
- **A[p U q]** (All paths Until): on all paths, p holds until q becomes true.
- **AG(p → AF q)** (AlwaysResponse): whenever p holds, q will eventually hold on all future paths.

The formula AG !p expresses that p is never reachable, while EF p indicates that p is reachable. The final verification step uses a model checker such as NuSMV, a symbolic model checker that supports both CTL and LTL specifications [14]. NuSMV takes as input a model description and a set of temporal logic formulas, and determines whether the model satisfies each formula. If a formula does not hold, NuSMV produces a counterexample trace showing

the execution path that violates the property, which is invaluable for debugging and understanding root causes of violations.

This pipeline enables automatic verification of privacy constraints by expressing them as reachability properties and evaluating them with NuSMV, providing a practical solution for ensuring privacy compliance in collaborative business processes. The following sections detail how this approach is extended to incorporate knowledge and duty tracking for choreography diagrams.

4 Requirements

This section describes the functional and non-functional requirements for the proposed framework. The requirements are derived from the goals outlined in the introduction and the gap analysis presented in the related work section.

4.1 Functional Requirements

The primary functional requirement is to extend the BPMVerification package with security-aware semantics for BPMN choreography diagrams. The extension must integrate seamlessly with the existing BPMVerification pipeline while adding the capability to verify privacy constraints.

4.1.1 Core Functionality

R1: Extension of BPMVerification Pipeline. The framework must extend the BPMVerification tool without modifying its core pipeline. The existing functionality for BPMN-to-Petri-net transformation, model checking integration, and control-flow verification must remain intact. All command-line flags from the user must be passed through to BPMVerification, except for tool-specific flags introduced by the extension.

R2: Privacy Specification Support. The framework must support a privacy specification file that allows users to define privacy constraints. The specification language must support three constraint types: Separation of Duties (SoD), Binding of Duties (BoD), and Division of Knowledge (DoK). The specification must be expressed in a machine-readable format (XML) that can be parsed and validated.

R3: Role-to-Transition Mapping. The framework must support mapping roles (participants) to transitions. Roles may be defined in the PNML file using existing tool-specific annotations, and the privacy specification may optionally provide additional mapping information. If a role is not specified for a transition, the framework must provide a fallback behavior (e.g., applying constraints globally).

R4: Modular Architecture. The framework must be modular, with each privacy concept implemented as an independent module. SoD, BoD, and DoK must be separate modules that can be loaded dynamically based on the constraints specified in the privacy file. This design reduces coupling and allows for future extensions.

R5: File Output. The framework must output two artifacts: (1) an augmented Petri net file (PNML) containing the original net augmented with privacy-related places and transitions, and (2) an augmented BPM specification file containing the privacy formulas. Both files must be valid and usable by the BPMVerification tool.

R6: Integration with BPMVerification. The framework must delegate formal verification to the BPMVerification package. After augmentation, the extended tool must invoke

the BPMVerification pipeline with the augmented net and augmented specification. The verification results must be displayed to the user.

4.2 Non-Functional Requirements

R7: Language Compatibility. The framework must be developed in Java to maintain compatibility with the BPMVerification package, which is also implemented in Java. The Java version must be compatible with the version used by BPMVerification.

R8: Command-Line Compatibility. The framework must accept the same command-line arguments as BPMVerification, with additional flags for privacy-specific options. The extended tool must be invocable from the command line using the same interface as the original tool.

R9: Backward Compatibility. The framework must support BPMN models without privacy annotations. When no privacy specification is provided, the tool must behave identically to the original BPMVerification tool, performing only control-flow verification.

R10: Extensibility. The modular architecture must support the addition of new privacy constraint types without modifying existing modules. New constraints should be implementable as new modules that integrate with the existing framework.

R11: Performance. The net augmentation must be efficient enough to handle realistic process models. The augmentation must not significantly increase the size of the net beyond what is necessary for privacy tracking. The Kripke structure size must remain manageable for model checking.

4.3 Design Requirements

R12: Technology Stack. The framework must use Java 17 or later, with Maven for build automation. Dependencies must be managed through Maven, including the BPMVerification package and BPMPetriNet library.

R13: JAXB for Specification Parsing. The privacy specification must be parsed using JAXB (Java Architecture for XML Binding) to ensure compatibility with the BPMVerification package and to simplify XML processing.

R14: Privacy Module Interface. All privacy modules must implement a common interface that defines the methods for net augmentation and specification augmentation. This ensures consistent behavior across modules and simplifies module loading.

4.4 Summary

The requirements outlined in this section establish the scope and constraints for the framework. The functional requirements define what the framework must do, the non-functional requirements define how it should perform, and the design requirements define the technical constraints. Together, these requirements guide the implementation described in Section 6.

5 Method

This section describes the method used to extend the BPMVerification tool with security-aware semantics. The approach is presented in three parts: the formal model underlying the extension, the privacy specification language, and the augmentation and verification pipeline.

5.1 Formal Model

The formal model extends the standard Petri net representation of BPMN choreography diagrams with constructs for tracking participant knowledge and duty states. Let P denote a finite set of participants, K a finite set of knowledge items, and D a finite set of duties. Each participant $p \in P$ is associated with a dynamic knowledge set $K_p \subseteq K$ and a dynamic duty set $D_p \subseteq D$. These sets may evolve during execution as interactions occur. Let T be a finite set of choreography tasks. Choreography tasks are enhanced with annotations specifying three types of semantics:

- **Required knowledge:** Knowledge items that must be possessed by the executing participant before the task can be performed.
- **Produced knowledge:** Knowledge items that are generated or transferred by the task and become possessed by the executing participant.
- **Assigned duties:** Duties that are assigned to the executing participant as a consequence of task execution.

For each task $t \in T$, we define three functions that formalize these annotations:

- *requiredKnowledge*: maps each task to the set of knowledge items required for execution.
- *producedKnowledge*: maps each task to the set of knowledge items produced by the task.
- *assignedDuties*: maps each task to the set of duties assigned as a consequence of execution.

To enable formal verification, the existing Petri net transformation of the BPMVerification tool is extended. In addition to the control-flow structure generated from the choreography model, the transformation introduces two types of places:

$$K_{p,k} \quad \text{for each } p \in P, k \in K \tag{1}$$

$$D_{p,d} \quad \text{for each } p \in P, d \in D \tag{2}$$

The place $K_{p,k}$ indicates that participant p possesses knowledge item k. The place $D_{p,d}$ indicates that duty d is currently assigned to participant p. Tokens in these places represent the possession of knowledge or the assignment of duties. Transitions corresponding to choreography tasks are augmented with arcs that reflect the annotated effects of the task. For a task $t \in T$ executed by participant p:

- For each $k \in \text{requiredKnowledge}(t)$: an arc is added from $K_{p,k}$ to t, representing consumption of the knowledge.
- For each $k \in \text{producedKnowledge}(t)$: an arc is added from t to $K_{p,k}$, representing production of the knowledge.
- For each $d \in \text{assignedDuties}(t)$: an arc is added from t to $D_{p,d}$, representing assignment of the duty.

Security constraints are then expressed as state predicates over the resulting Petri net. Three types of constraints are supported:

1. **Separation of Duties (SoD):** A constraint $c = (p, \text{conflictingDuties})$ is violated if there exists a reachable marking M such that for all $d \in \text{conflictingDuties}$, $M(D_{p,d}) \geq 1$.
2. **Division of Knowledge (DoK):** A constraint $c = (p, \text{criticalSet})$ is violated if there exists a reachable marking M such that for all $k \in \text{criticalSet}$, $M(K_{p,k}) \geq 1$.
3. **Binding of Duties (BoD):** A constraint $c = (p, \text{dutySet})$ is satisfied if there exists a reachable marking M such that for all $d \in \text{dutySet}$, $M(D_{p,d}) \geq 1$.

Verification is performed by checking the reachability of such undesirable markings using the model checker. Example depicted in Figure 3.

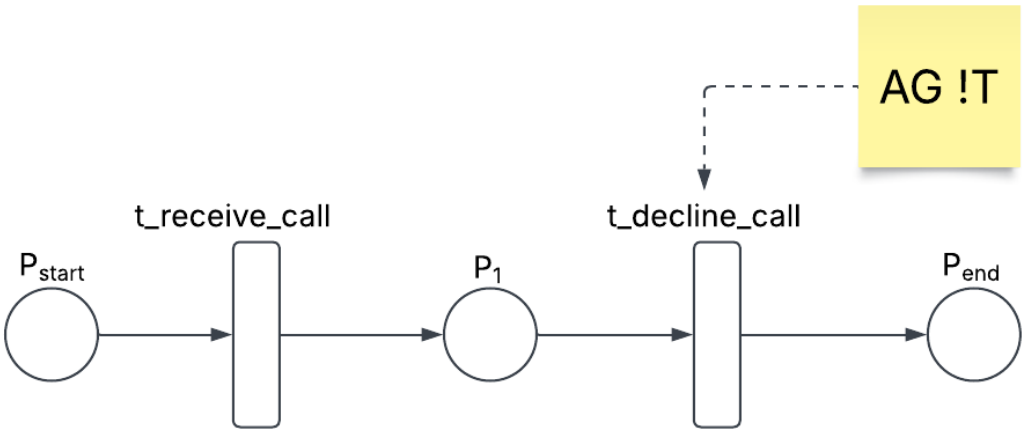

Figure 3: Reachability check example.

5.2 Privacy Specification Language

The framework introduces a privacy specification language expressed in XML. This language allows users to define the privacy constraints to be verified. The specification file is passed to the tool using an additional command-line flag. The XML schema defines four main sections:

5.2.1 Role-to-Transition Mapping

The role mapping section associates participants with the transitions they execute. This mapping can also be derived from the PNML tool-specific sections, but the privacy specification provides an explicit mechanism.

```
<roleMappings>
    <role id="Alice">
        <transition>t_create_order</transition>
        <transition>t_approve_order</transition>
    </role>
</roleMappings>
```

5.2.2 Separation of Duties

The SoD section defines duties and the transitions that create them, followed by constraints that specify conflicting duty sets.

```
<separationOfDuty>
    <duties>
        <duty id="duty_create_order">
            <createdBy>t_create_order</createdBy>
        </duty>
    </duties>
    <constraints>
        <constraint id="sod1">
            <conflictingDuties>
                <duty>duty_create_order</duty>
                <duty>duty_approve_order</duty>
            </conflictingDuties>
        </constraint>
    </constraints>
</separationOfDuty>
```

5.2.3 Binding of Duties

The BoD section defines duties and the transitions that create them, followed by constraints that specify which duties must be bound together (assigned to the same participant).

```
<bindingOfDuty>
    <duties>
        <duty id="duty_approve_order">
            <createdBy>t_approve_order</createdBy>
        </duty>
        <duty id="duty_approve_payment">
            <createdBy>t_approve_payment</createdBy>
```

```
        </duty>
    </duties>
    <constraints>
        <constraint id="bod1">
            <boundDuties>
                <duty>duty_approve_order</duty>
                <duty>duty_approve_payment</duty>
            </boundDuties>
        </constraint>
    </constraints>
</bindingOfDuty>
```

5.2.4 Division of Knowledge

The DoK section defines knowledge items, the transitions that produce them, the transitions that require them, and constraints that specify critical knowledge sets.

```
<divisionOfKnowledge>
    <knowledgeSet>
        <knowledge id="contract_number">
            <createdBy>t_task_a</createdBy>
            <requiredBy>t_task_b</requiredBy>
        </knowledge>
        <knowledge id="payment_info">
            <createdBy>t_task_c</createdBy>
        </knowledge>
    </knowledgeSet>
    <constraints>
        <constraint id="dok1">
            <role>Alice</role>
            <criticalSet>
                <knowledge>contract_number</knowledge>
                <knowledge>payment_info</knowledge>
            </criticalSet>
        </constraint>
    </constraints>
</divisionOfKnowledge>
```

5.3 Augmentation and Verification Pipeline

5.3.1 Stage 1: Specification Parsing

The tool first loads the BPMN model and the BPMVerification configuration. It then parses the privacy specification file using JAXB (Java Architecture for XML Binding), producing an instance of the `BPMNPrivacySpec` class. This class contains all the privacy constraints defined by the user.

5.3.2 Stage 2: Module Instantiation

The `PrivacyModuleFactory` receives the privacy specification and instantiates the required privacy modules. The factory examines the specification and creates modules for each constraint type present:

- If the specification contains a `separationOfDuty` section, an `SoDModule` is instantiated.
- If the specification contains a `divisionOfKnowledge` section, a `DoKModule` is instantiated.
- Future constraint types can be added by implementing additional modules.

All privacy modules implement the `IPrivacyModule` interface, which defines three methods:

1. `augmentPNML(PlaceTransitionNet net)`: Augments the Petri net with privacy-related places and transitions.
2. `augmentSpecification(BPMSpecification spec)`: Augments the BPM specification with Specification Sets (that are eventually translated to CTL formulas).
3. `apply(PlaceTransitionNet net, BPMSpecification spec)`: Calls both augmentation methods in sequence.

The modules are decoupled and independent. The user may have any number of privacy modules, and they will not conflict with each other.

5.3.3 Stage 3: Net Augmentation

For each instantiated module, the `augmentPNML` method is called. This method modifies the Petri net by adding:

1. **Duty places:** For SoD constraints, places are created for each participant-duty pair.
2. **Knowledge places:** For DoK constraints, places are created for each participant-knowledge pair.
3. **Violation detectors:** For each constraint, a transition is added that fires when the violation condition is met. These transitions consume tokens from the relevant duty or knowledge places.

The augmented net retains all original control-flow structures while adding the privacy-aware elements. Figure 4 shows an example of the augmentation (depicted in red).

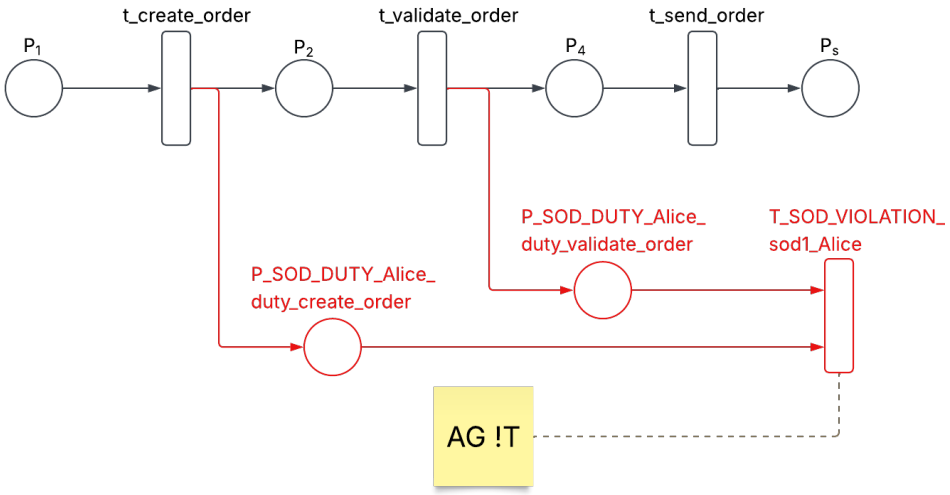

Figure 4: Example of net augmentation.

5.3.4 Stage 4: Specification Augmentation

The `augmentSpecification` method is called for each module. This method augments the BPM specification with CTL formulas corresponding to the privacy constraints:

- **SoD and DoK:** The formula $AG\neg detector$ is added, expressing that the violation detector is never enabled.
- **BoD:** The formula $AG(t \rightarrow AF detector)$ is added for each transition that is creating a duty, expressing that whenever a participant performs a task that creates a bound duty, the remaining duties in the set must eventually also be performed by the same participant.

5.3.5 Stage 5: Export and Verification

After augmentation, the framework exports two artifacts:

1. **Augmented PNML file:** The augmented Petri net is written to a file with the prefix `augmented_`.
2. **Augmented specification file:** The augmented BPM specification is written to a file with the prefix `augmented_`.

Both the input net and the augmented net can be viewed side by side, facilitating debugging and understanding of the program flow. Finally, the tool calls the regular BPMVerification verification pipeline using the helper class, with the augmented Petri net and augmented specification as input. The privacy specification file is used solely for the privacy verification tool, maintaining a clear separation of concerns.

6 Implementation

This section describes the implementation of the framework. The implementation extends the BPMVerification package with privacy-aware semantics, following the method outlined in Section 5. The implementation is organized into several components: the JAXB classes for parsing the privacy specification, the privacy module interface and factory, the individual privacy modules (SoD, BoD, and DoK), the net and specification augmentation logic, and the main program flow that orchestrates the entire pipeline.

6.1 Architecture Overview

The implementation follows a modular architecture, as illustrated in Figure 5. The system is organized into three layers: the specification parsing layer, the augmentation layer, and the verification layer. The specification parsing layer is responsible for reading and validating the privacy specification file. The augmentation layer contains the privacy modules that extend the Petri net and the BPM specification. The verification layer delegates the formal verification to the BPMVerification package.

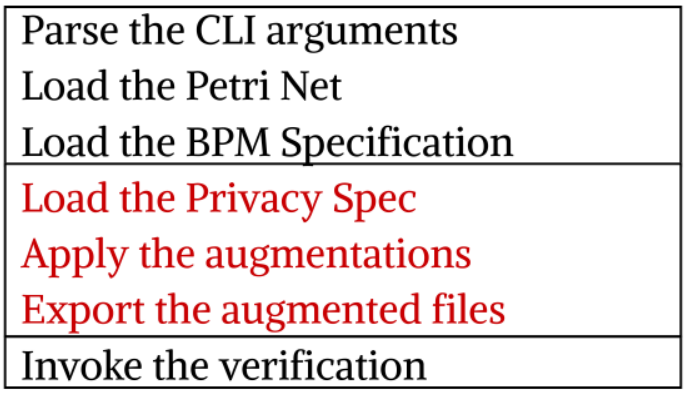

Figure 5: The layer structure of the tool.

The entry point is the `Main` class, which handles command-line parsing, loads the input artifacts, and orchestrates the verification pipeline. The `PrivacyModuleFactory` is responsible for discovering and instantiating privacy modules based on the contents of the privacy specification. Each privacy module implements the `IPrivacyModule` interface, allowing them to be loaded and executed uniformly. The `AugmentedPTNetMarshaller` exports the augmented Petri net to PNML format, and the `SpecificationMarshaller` exports the augmented BPM specification to XML format.

6.2 JAXB Classes for Privacy Specification

The privacy specification is parsed using JAXB (Java Architecture for XML Binding). The root class is `BPMNPrivacySpec`, which contains the three privacy modules: separation of

duty, division of knowledge, and binding of duty. Each module is represented by its own JAXB class.

```
@XmlRootElement(name = "privacySpecification")
@XmlAccessorType(XmlAccessType.FIELD)
public class BPMNPrivacySpec {
    @XmlElementWrapper(name = "roleMappings")
    @XmlElement(name = "role")
    private List<RoleMapping> roleMappings;

    @XmlElement(name = "separationOfDuty")
    private SeparationOfDuty separationOfDuty;

    @XmlElement(name = "divisionOfKnowledge")
    private DivisionOfKnowledge divisionOfKnowledge;

    @XmlElement(name = "bindingOfDuty")
    private BindingOfDuty bindingOfDuty;
}
```

The `RoleMapping` class associates a role (participant) with the transitions it executes. This mapping is used to determine which participant executes which tasks, which is essential for tracking knowledge and duty states.

Each privacy module follows a similar structure. For example, `SeparationOfDuty` contains a list of duties and a list of constraints. Each `Duty` defines a duty identifier and the transitions that create it. Each `Constraint` defines a set of conflicting duties that must not be held simultaneously by the same participant.

6.3 Privacy Module Interface and Factory Pattern

All privacy modules implement the `IPrivacyModule` interface, which defines the contract for all modules. The interface consists of three methods:

```
public interface IPrivacyModule {
    void apply(PlaceTransitionNet net, BPMSpecification spec);
    void augmentPNML(PlaceTransitionNet net) throws MalformedNetException;
    void augmentSpecification(BPMSpecification spec);
}
```

The `apply` method orchestrates the augmentation process by calling both `augmentPNML` and `augmentSpecification` in sequence. This design ensures that all modules follow a consistent augmentation pattern.

The `AbstractPrivacyModule` class provides a default implementation of the `apply` method, allowing concrete modules to focus on their specific augmentation logic:

```
public abstract class AbstractPrivacyModule implements IPrivacyModule {
    @Override
    public void apply(PlaceTransitionNet net, BPMSpecification spec) {
        augmentPNML(net);
        augmentSpecification(spec);
    }
}
```

The `PrivacyModuleFactory` is responsible for instantiating the appropriate modules based on the privacy specification. The factory examines the `BPMNPrivacySpec` object and creates modules only for the sections that are present. For example, if the specification contains a `separationOfDuty` section, an `SoDModule` is instantiated and configured with the relevant data.

```
public class PrivacyModuleFactory {
    public static List<IPrivacyModule> loadModules(
        PlaceTransitionNet net,
        BPMSpecification specification,
        String privacySpecPath
    ) throws Exception {
        List<IPrivacyModule> modules = new ArrayList<>();

        BPMNPrivacySpec privacySpec = parsePrivacySpec(privacySpecPath);

        if (privacySpec.getSeparationOfDuty() != null) {
            SoDModule sodModule = new SoDModule();
            sodModule.setSeparationOfDuty(privacySpec.getSeparationOfDuty());
            sodModule.setRoleMappings(privacySpec.getRoleMappings());
            modules.add(sodModule);
        }
        // Similar for DoK and BoD modules
        return modules;
    }
}
```

6.4 SoD Module Implementation

The `SoDModule` implements the Separation of Duties verification. The module modifies the Petri net by adding duty places and violation detectors. The augmentation process follows these steps:

1. Parse the `separationOfDuty` section from the privacy specification.
2. For each constraint, identify which roles execute the duties involved.
3. For each duty, create a duty place: `P_SOD_DUTY_{role}_{duty}`.

4. Connect the creating transitions to the duty place using arcs.
5. Create a violation detector transition: `T_SOD_VIOLATION_{constraint}_{role}`.
6. Connect all duty places in the constraint to the violation detector as inputs.
7. Store the violation detector name for specification generation.

Figure 6 illustrates the augmentation process for an SoD constraint. The original control-flow net is extended with duty places and a violation detector.

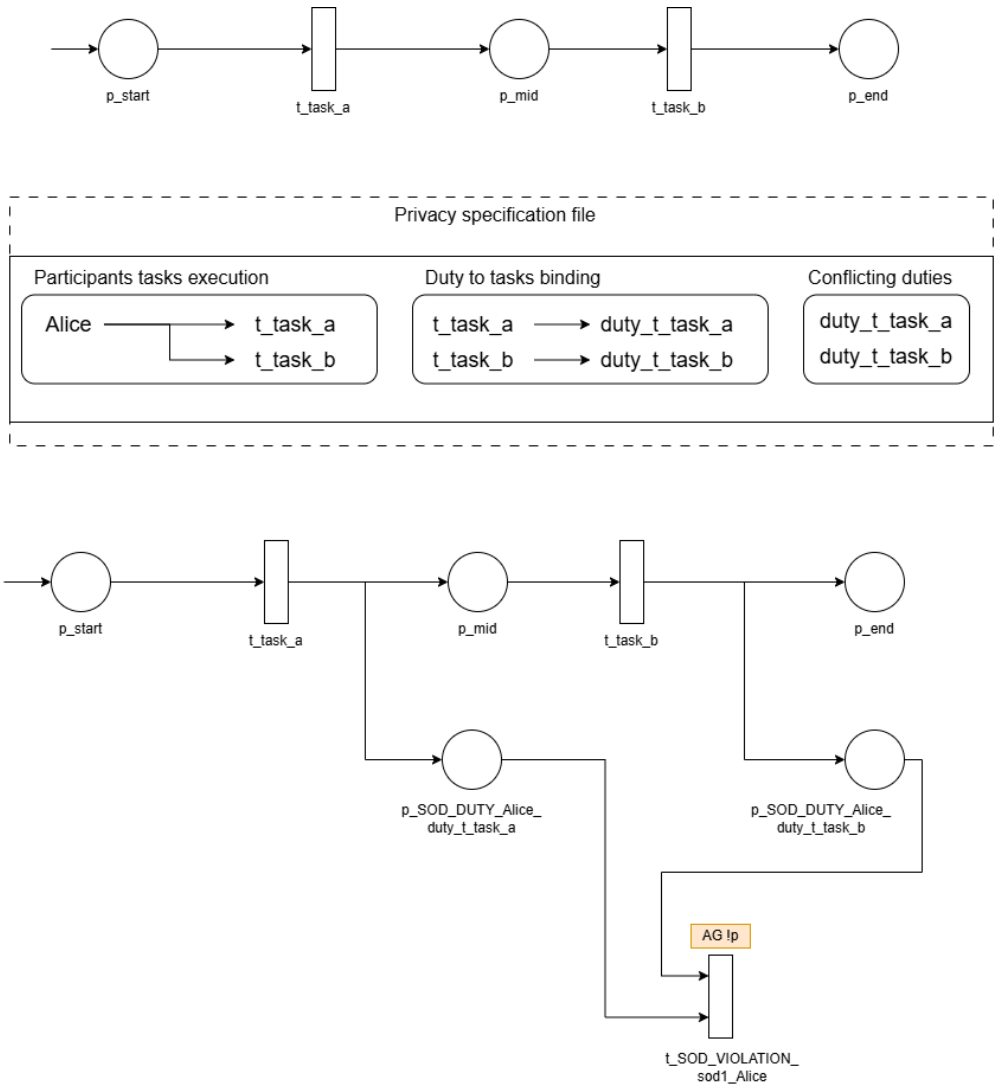

Figure 6: Augmentation for an SoD constraint.

The key method in the SoD module is `createViolationDetector`, which performs the actual net augmentation:

```
private void createViolationDetector(
    PlaceTransitionNet net,
    Constraint constraint,
    String roleId
) {
        Map<String, Place> dutyPlaces = new HashMap<>();
        Set<String> roleTransitions = getTransitionsForRole(roleId);

        for (String dutyId : constraint.getConflictingDuties()) {
            Duty duty = findDutyById(dutyId);
            if (duty == null) continue;

            boolean roleExecutesDuty = false;
            for (String transitionId : duty.getCreatedBy()) {
                if (roleTransitions.contains(transitionId)) {
                    roleExecutesDuty = true;
                    break;
                }
            }

            if (!roleExecutesDuty) continue;

            String placeName = "P_SOD_DUTY_" + constraint.getId() + "_" + roleId
            + "_" + dutyId;
            Place dutyPlace = new Place(placeName);
            net.addPlace(dutyPlace);
            dutyPlaces.put(dutyId, dutyPlace);

            for (String transitionId : duty.getCreatedBy()) {
                if (roleTransitions.contains(transitionId)) {
                    Transition transition = net.getTransition(transitionId);
                    if (transition != null) {
                        net.addArc(transition, dutyPlace);
                    }
                }
            }
        }

        if (dutyPlaces.size() >= 2) {
            String detectorName = "T_SOD_VIOLATION_"
            + constraint.getId() + "_" + roleId;
            Transition violationDetector = new Transition(detectorName);
            net.addTransition(violationDetector);
            violationDetectors.add(detectorName);
```

```
            for (Place dutyPlace : dutyPlaces.values()) {
                net.addArc(dutyPlace, violationDetector);
            }
        }
    }
```

6.5 DoK Module Implementation

The `DoKModule` implements the Division of Knowledge verification. The module extends the Petri net with knowledge places and violation detectors that fire when a participant accumulates all knowledge items in a critical set. The augmentation process follows these steps:

1. Parse the `divisionOfKnowledge` section from the privacy specification.
2. For each constraint, identify the role and its critical knowledge set.
3. For each knowledge item in the critical set, create a knowledge place: `P_KNOWLEDGE_{role}_{knowledge}`.
4. Connect transitions that produce the knowledge to the knowledge place: `transition` $\rightarrow$ `knowledgePlace`.
5. Connect transitions that require the knowledge from the knowledge place: `knowledgePlace` $\rightarrow$ `transition`.
6. Create a violation detector transition: `T_DOK_VIOLATION_{constraint}_{role}`.
7. Connect all knowledge places in the critical set to the violation detector as inputs.
8. Store the violation detector name for specification generation.

The knowledge tracking distinguishes between production and consumption of knowledge. When a transition produces knowledge, a token is added to the knowledge place. When a transition requires knowledge, a token is consumed from the knowledge place. This accurately models the dynamic knowledge state of participants.

6.6 BoD Module Implementation

The `BoDModule` implements the Binding of Duties verification. The module is structurally similar to SoD but uses a different semantics. While SoD detects violations when conflicting duties are held, BoD detects when duties that must be bound together are held by the same participant. The augmentation process for BoD is analogous to SoD, but the verification formula differs. BoD constraints are checked using the CTL formula $AG(t \rightarrow AF\ q)$, which expresses that whenever a transition t creates a duty belonging to a bound set, the binding detector q must eventually be reached, ensuring that all the bounded duties are performed by the same participant.

6.7 Specification Augmentation

The specification augmentation process adds CTL formulas to the BPM specification. For each violation detector created, the corresponding module adds a specification that checks whether the detector is enabled. For SoD and DoK constraints, the formula $AG\neg detector$ is added. This formula expresses that the violation detector is never enabled, meaning the violation is unreachable. For BoD constraints, the formula $AG(t \rightarrow AF\ q)$ (known as *AlwaysResponse*) is added, expressing that the binding is held.

```
// In SoDModule.augmentSpecification()
for (String detector : violationDetectors) {
    Group group = new Group();
    group.setId("group_" + detector);

    Element element = new Element();
    element.setId(detector);
    group.getElements().add(element);
    spec.addGroup(group);

    Specification specification = new Specification();
    specification.setId("check_" + detector);
    specification.setType("SoDReachability");

    InputElement inputElement = new InputElement();
    inputElement.setTarget("p");
    inputElement.setElement(group.getId());
    specification.getInputElements().add(inputElement);
}
```

6.8 Net and Specification Export

The augmented artifacts are exported using custom marshallers. The `AugmentedPTNetMarshaller` class exports the augmented Petri net to PNML format. It extracts the JAXB representation of the net and marshals it to a file.

```
public class AugmentedPTNetMarshaller {
    public AugmentedPTNetMarshaller(PlaceTransitionNet net, File file) {
        Net jaxbNet = (Net) net.getXmlElement();
        Pnml pnml = new Pnml();
        Set<Net> nets = new HashSet<>();
        nets.add(jaxbNet);
        pnml.setNets(nets);
    }
}
```

The augmented specification is exported using the existing `SpecificationMarshaller` from the BPMVerification package.

6.9 Main Program Flow

The main program flow orchestrates the entire pipeline. The `Main` class handles command-line parsing, loads the input artifacts, invokes the privacy modules, and delegates verification to BPMVerification.
The key steps are:

1. Parse command-line arguments using Apache Commons CLI.
2. Load the Petri net using the BPMVerification helper.
3. Load the BPM specification using the BPMVerification helper.
4. Parse the privacy specification using JAXB.
5. Load privacy modules using the `PrivacyModuleFactory`.
6. For each module, call `apply` to augment the net and specification.
7. Export the augmented net to `augmented.pnml`.
8. Export the augmented specification to `augmented.spec`.
9. Invoke the BPMVerification verification pipeline with the augmented artifacts.

A clear separation of responsibilities is maintained throughout the implementation. The tool is responsible for privacy-specific concerns: parsing the privacy specification, augmenting the Petri net with privacy-aware places and transitions, and generating the corresponding CTL formulas. The loading of artifacts, transformation of BPMN to Petri nets, and the actual model checking are delegated to the BPMVerification helper. This separation ensures that the core verification pipeline remains untouched, while the extension seamlessly integrates privacy verification into the existing workflow. By keeping these concerns distinct, the implementation remains maintainable, testable, and extensible to future privacy constraint types.

7 Evaluation

This section evaluates the proposed framework through a set of test cases designed to demonstrate its effectiveness in verifying privacy constraints in BPMN choreography diagrams. The evaluation is structured around three privacy constraint types: Separation of Duties (SoD), Binding of Duties (BoD), and Division of Knowledge (DoK). Each test case examines whether the framework correctly detects reachable violations and correctly identifies unreachable violations. The section concludes with a discussion of the findings, limitations, and insights gained from the evaluation.

7.1 Test Setup

The evaluation of the proposed framework was performed on a standard development PC running Windows, with sufficient memory and CPU resources to handle Petri net generation and model checking tasks. Software tools include an IDE (IntelliJ), the Java runtime environment, Git Version Control and the BPMVerification tool [8], which is forked and extended for the purposes of this thesis. No specialized hardware is required.
The evaluation was conducted using the BPMVerification package v1.2.0 on a standard development machine running Windows 10. The tool was configured to use NuSMV as the underlying model checker and Kripke structure-based verification. The test suite consists of three categories of tests, each corresponding to a privacy constraint type. For each test case, the following artifacts were prepared:

- A BPMN choreography diagram (in PNML format) representing the process under test.
- A BPM specification file defining the behavioral properties of the process.
- A privacy specification file defining the privacy constraints to be verified.

The tool was invoked with the privacy specification file, producing an augmented Petri net and an augmented BPM specification. The verification results were then analyzed to determine whether the privacy constraints were correctly evaluated.

7.2 Separation of Duties Test Case

The SoD test case evaluates whether the framework correctly detects violations where a participant holds conflicting duties. The petri net diagram is presented in Figure 7 (here black color represents - the original petri net semantics, red color - the first SoD constraint augmentations and blue color - the second SoD constraint augmentations).

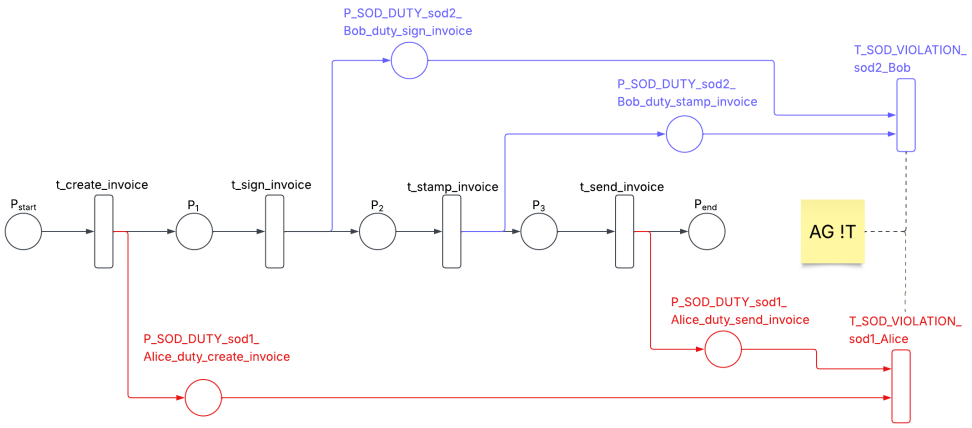

Figure 7: Separation of Duties Test Case.

The resulting petri net was obtained using the following privacy specification file:

```
<privacySpecification>

    <!-- Role Mapping -->
    <roleMappings>
        <role id="Alice">
            <transition>t_create_invoice</transition>
            <transition>t_send_invoice</transition>
        </role>
        <role id="Bob">
            <transition>t_sign_invoice</transition>
            <transition>t_stamp_invoice</transition>
        </role>
    </roleMappings>

    <!-- SoD Module -->
    <separationOfDuty>
        <duties>
            <duty id="duty_create_invoice">
                <createdBy>
                    <transition>t_create_invoice</transition>
                </createdBy>
            </duty>
            <duty id="duty_sign_invoice">
                <createdBy>
                    <transition>t_sign_invoice</transition>
                </createdBy>
            </duty>
            <duty id="duty_stamp_invoice">
```

```
                <createdBy>
                    <transition>t_stamp_invoice</transition>
                </createdBy>
            </duty>
            <duty id="duty_send_invoice">
                <createdBy>
                    <transition>t_send_invoice</transition>
                </createdBy>
            </duty>
        </duties>

        <constraints>
            <constraint id="sod1">
                <conflictingDuties>
                    <duty>duty_create_invoice</duty>
                    <duty>duty_send_invoice</duty>
                </conflictingDuties>
            </constraint>

            <constraint id="sod2">
                <conflictingDuties>
                    <duty>duty_sign_invoice</duty>
                    <duty>duty_stamp_invoice</duty>
                </conflictingDuties>
            </constraint>
        </constraints>

    </separationOfDuty>
</privacySpecification>
```

The model checking result correctly concludes that both SoD constraints fail because both participants execute all the conflicting duties. The console output can be observed in Figure 8.

```
Arcs in net: 16
PNML successfully written to augmented.pnml
==============================

[2026-07-06 17:15:26] INFO:     Loading configuration file
[2026-07-06 17:15:26] INFO:     Verifying specification sets
[2026-07-06 17:15:26] INFO:     Verifying set.
[2026-07-06 17:15:26] INFO:     Calculating Kripke structure
[2026-07-06 17:15:26] INFO:     Calculated Kripke structure with |S| = 9, |R| = 11, |AP| = 7
 in 12.598 ms
[2026-07-06 17:15:26] INFO:     Collecting specifications
[2026-07-06 17:15:26] INFO:     Calling Model Checker
[2026-07-06 17:15:26] FEEDBACK: Specification AG !group_T_SOD_VIOLATION_sod1_Alice with id check_T_SOD_VIOLATION_sod1_Alice failed because SOD Violation is reachable. given the fol
lowing counter example: {t_create_invoice} -> {t_sign_invoice} -> {t_stamp_invoice} -> {group_T_SOD_VIOLATION_sod2_Bob,t_send_invoice} -> {group_T_SOD_VIOLATION_sod2_Bob,group_T_SO
D_VIOLATION_sod1_Alice,T_SOD_VIOLATION_sod1_Alice}
[2026-07-06 17:15:26] INFO:     Specification check_T_SOD_VIOLATION_sod1_Alice evaluated false for AG !group_T_SOD_VIOLATION_sod1_Alice
[2026-07-06 17:15:26] FEEDBACK: Specification AG !group_T_SOD_VIOLATION_sod2_Bob with id check_T_SOD_VIOLATION_sod2_Bob failed because SOD Violation is reachable. given the followi
ng counter example: {t_create_invoice} -> {t_sign_invoice} -> {t_stamp_invoice} -> {group_T_SOD_VIOLATION_sod2_Bob,t_send_invoice}
[2026-07-06 17:15:26] INFO:     Specification check_T_SOD_VIOLATION_sod2_Bob evaluated false for AG !group_T_SOD_VIOLATION_sod2_Bob
Metrics for net: net1 -
Specification set:
- Conditions:
- Specifications: check_T_SOD_VIOLATION_sod1_Alice, check_T_SOD_VIOLATION_sod2_Bob

StructureComputationMs  =       12.5983
StructureStateCount     =       9
StructureRelationCount  =       11
StructureAtomicPropositionCount =       7

[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  1.046 s
[INFO] Finished at: 2026-07-06T17:15:26+03:00
[INFO] ------------------------------------------------------------------------
```

Figure 8: Separation of Duties console output.

7.3 Division of Knowledge Test Case

The DoK test case evaluates whether the framework correctly detects violations where a participant holds all knowledge items in a critical set. The resulting petri net can be observed in Figure 9 (here black color represents - the original petri net semantics, red color - the DoK constraint augmentations).

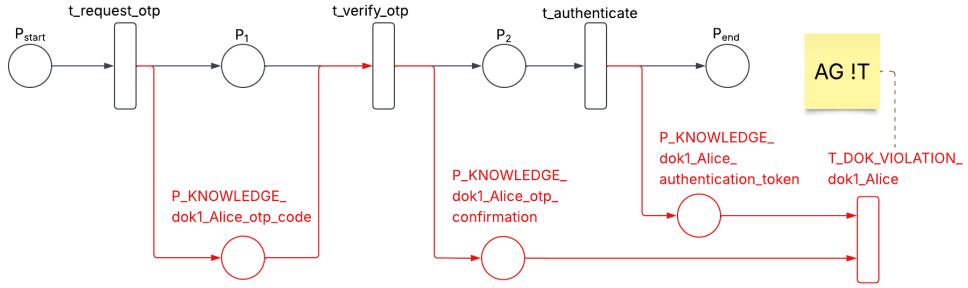

Figure 9: Division of Knowledge Test Case.

The resulting petri net was obtained using the following privacy specification file:

```
<privacySpecification>

    <!-- Role Mapping -->
    <roleMappings>
        <role id="Alice">
            <transition>t_request_otp</transition>
            <transition>t_verify_otp</transition>
            <transition>t_authenticate</transition>
```

```
        </role>
    </roleMappings>

    <!-- DoK Module -->
     <divisionOfKnowledge>
         <knowledgeSet>
             <knowledge id="otp_code">
                 <createdBy>
                     <transition>t_request_otp</transition>
                 </createdBy>
                 <requiredBy>
                     <transition>t_verify_otp</transition>
                 </requiredBy>
             </knowledge>
             <knowledge id="otp_confirmation">
                 <createdBy>
                     <transition>t_verify_otp</transition>
                 </createdBy>
             </knowledge>
             <knowledge id="authentication_token">
                 <createdBy>
                     <transition>t_authenticate</transition>
                 </createdBy>
             </knowledge>
         </knowledgeSet>

         <constraints>
             <constraint id="dok1">
                 <role>Alice</role>
                 <criticalSet>
                     <knowledge>otp_confirmation</knowledge>
                     <knowledge>authentication_token</knowledge>
                 </criticalSet>
             </constraint>
         </constraints>

        </divisionOfKnowledge>
</privacySpecification>
```

The model checking result correctly concludes that the DoK constraint fails because the participant possesses all critical knowledge pieces. The console output can be observed in Figure 10.

Figure 10: Division of Knowledge console output.

7.4 Binding of Duties Test Case

The BoD test case evaluates whether the framework correctly detects violations where a participant does not execute a set of bounded duties. The petri net diagram is presented in Figure 11 (here black color represents - the original petri net semantics, red color - the first BoD constraint augmentations and blue color - the second BoD constraint augmentations).

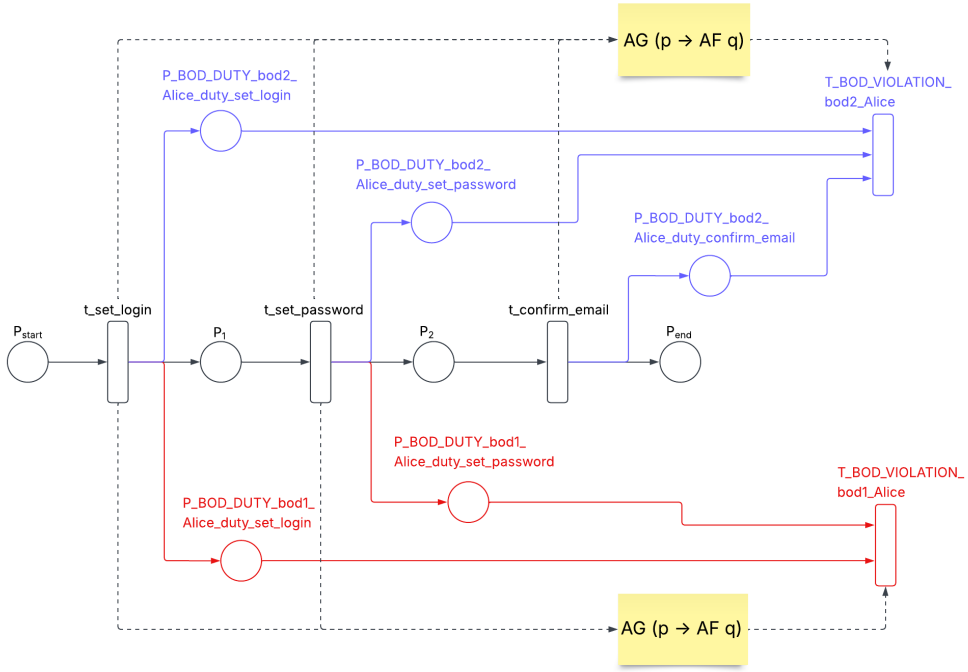

Figure 11: Binding of Duties Test Case.

The resulting petri net was obtained using the following privacy specification file:

```
<privacySpecification>

    <!-- Role Mapping -->
    <roleMappings>
        <role id="Alice">
            <transition>t_set_login</transition>
            <transition>t_set_password</transition>
            <transition>t_confirm_email</transition>
        </role>
    </roleMappings>

    <!-- BoD Module -->
    <bindingOfDuty>
        <duties>
            <duty id="duty_set_login">
                <createdBy>
                    <transition>t_set_login</transition>
                </createdBy>
            </duty>
            <duty id="duty_set_password">
                <createdBy>
                    <transition>t_set_password</transition>
```

```
                </createdBy>
            </duty>
            <duty id="duty_confirm_email">
                <createdBy>
                    <transition>t_confirm_email</transition>
                </createdBy>
            </duty>
        </duties>

        <constraints>
            <constraint id="bod1">
                <boundDuties>
                    <duty>duty_set_login</duty>
                    <duty>duty_set_password</duty>
                </boundDuties>
            </constraint>

            <constraint id="bod2">
                <boundDuties>
                    <duty>duty_set_login</duty>
                    <duty>duty_set_password</duty>
                    <duty>duty_confirm_email</duty>
                </boundDuties>
            </constraint>
        </constraints>

    </bindingOfDuty>
</privacySpecification>
```

The model checking result correctly concludes that both BoD constraint hold because the participants execute all the bounded duties. The console output can be observed in Figure 12.

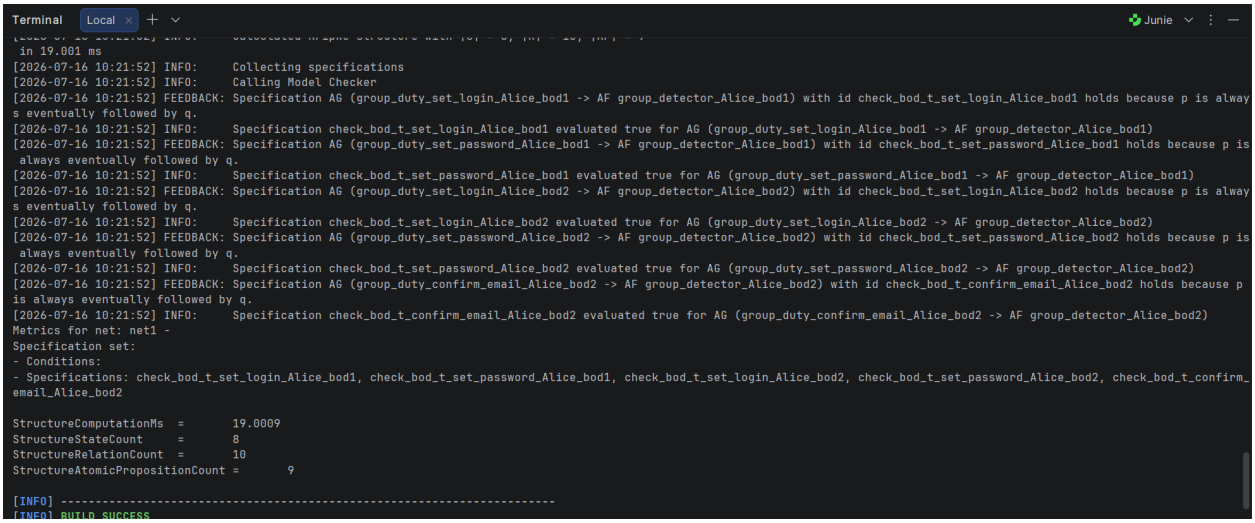

Figure 12: Binding of Duty console output.

7.5 Discussion

The evaluation results demonstrate that the framework successfully verifies privacy constraints in BPMN choreography diagrams. The following key observations can be made:

- First, correct net augmentation is critical; once the augmentation accurately models the privacy semantics, the specification generation becomes straightforward, and the exported augmented net greatly facilitates debugging.
- Second, counterexamples generated by NuSMV provide clear execution traces that are valuable for understanding why violations are reachable or unreachable and for validating the correctness of the augmentation.

7.6 Summary of Evaluation Findings

The evaluation confirms that the proposed framework successfully extends the BPMVerification tool with privacy-aware semantics. The framework correctly detects reachable violations for SoD, DoK, and BoD constraints, and correctly identifies unreachable violations.

8 Conclusion

8.1 Ethical Considerations

The primary stakeholders of this thesis are the academic community, including the University of Groningen, and the authors of the BPMVerification tool, whose codebase is extended with their permission. All modifications are intended solely for research purposes, with no commercial use planned. Privacy implications do not exceed those already present in the original framework, and all contributions are properly cited. No public or private release of the extended tool or results will occur without the authors' explicit consent, ensuring ethical standards regarding software use, authorship acknowledgment, and privacy are fully respected.

8.2 Summary of Contributions

This thesis has presented an extension to the BPMVerification tool that enables the automated verification of privacy constraints in BPMN choreography diagrams. The main contributions are:

1. A formal model for tracking participant knowledge and duty states has been defined. The model introduces places for each participant-knowledge pair $K_{p,k}$ and each participant-duty pair $D_{p,d}$, extending the standard Petri net representation with privacy-aware semantics. This formalization enables privacy constraints to be expressed as reachability properties and verified using standard model-checking techniques.

2. The BPMVerification tool has been extended with a modular architecture that supports the dynamic loading of privacy modules. The SoD, BoD, and DoK modules have been implemented as independent components, demonstrating the extensibility of the framework. This modular design allows for the addition of new privacy constraint types without modifying existing code.

3. A privacy specification language has been developed that allows users to define privacy constraints in a machine-readable XML format. The specification language supports role-to-transition mappings, duty definitions, knowledge definitions, and constraint definitions. The language is designed to be extensible, allowing for the addition of new constraint types in the future.

4. The framework has been evaluated on a set of test cases, demonstrating that privacy constraints can be verified alongside behavioral properties. The evaluation confirms that the approach is feasible and that the augmented nets and specifications are correctly generated.

8.3 Findings and Discussion

- The results of this thesis indicate that privacy concepts can be efficiently and formally verified using existing work in the domain. The key insight is that once the Petri net

augmentation is done correctly, the specification generation is often straightforward. Most privacy constraints require relatively simple CTL formulas, with the complexity concentrated in the net augmentation phase. The biggest challenge identified in this work is performing the Petri net augmentation correctly. The augmentation must accurately reflect the intended privacy semantics, including the creation of duty and knowledge places, the connection of transitions to these places, and the addition of violation detectors. Debugging the augmentation is facilitated by the tool's ability to output the augmented net file, which can be compared with the initial net to verify correctness. The counterexample generation provided by the model checker proved to be a valuable debugging tool. When a violation was detected, the counterexample trace clearly showed the sequence of transitions that led to the violation, making it easy to understand and fix the underlying issue.

- Separation of Duties (SoD), Binding of Duties (BoD), and Division of Knowledge (DoK)—can be meaningfully defined for BPMN choreographies through task annotations and role mappings. These conditions can be operationally represented using Petri nets by introducing dedicated places for knowledge and duty states, and by augmenting transitions with arcs that reflect the security-relevant effects of each task, and the verification of these constraints can be automated alongside standard behavioral checks by extending the BPMVerification tool with privacy modules that perform net augmentation and specification generation, and by invoking the existing model checking pipeline.

8.4 Applicability to Other Privacy Concepts

The approach presented in this thesis can be extended to verify a range of other privacy and security concepts. The following examples illustrate the flexibility of the framework:

- **Chinese Wall Policy.** The Chinese Wall policy is a security model that prevents participants from accessing data that could create a conflict of interest. This can be modeled by defining groups of knowledge items that must not be held simultaneously by the same participant. The verification approach is similar to DoK, with violation detectors firing when a participant holds knowledge from conflicting groups.

- **Need-to-Know Principle.** The need-to-know principle states that participants should only have access to the minimum information necessary to perform their tasks. This can be verified by checking that participants do not hold knowledge items that are not required for their assigned duties. The framework can be extended to track task assignments and verify that knowledge sets are limited to the required set.

- **Role-Based Access Control (RBAC).** RBAC policies can be verified by ensuring that participants are only assigned duties consistent with their roles. The framework can be extended to support role assignments and check that duty assignments respect role-based permissions.

- **Compliance with Regulations.** Many regulations (such as GDPR or HIPAA) impose constraints on how sensitive information is handled. The framework can be extended to model these constraints as privacy properties, enabling automated compliance checking.

8.5 Limitations

While the framework successfully demonstrates the feasibility of privacy verification in BPMN choreographies, several limitations should be acknowledged.

1. The framework currently relies on annotations added to the choreography model. This requires modelers to explicitly specify knowledge requirements, produced knowledge, and duties. While this is necessary for formal verification, it adds a modeling overhead that might be burdensome in practice.

2. The framework currently supports only SoD, BoD, and DoK constraints. While these are important privacy concepts, other privacy properties such as anonymity, unlinkability, and transparency are not covered. This limitation is primarily due to the scope and timeframe of this thesis.

3. The inference-aware version of DoK, where participants can infer knowledge from other knowledge items, is not supported. This is a more advanced form of DoK that would require additional modeling of inference rules.

8.6 Future Work

Several directions for future research arise from this work:

- **Inference-Aware DoK.** Extending the framework to support inference-aware DoK would involve modeling the inference rules that participants can apply to derive new knowledge from existing knowledge. This would require a more sophisticated knowledge representation and reasoning engine.

- **Integration with Other Verification Tools.** The framework could be extended to support other model checkers and verification tools, providing flexibility in the choice of verification engine.

- **Graphical Privacy Specification.** A graphical interface for defining privacy specifications could lower the barrier to adoption and make the tool more accessible to non-technical users.

- **Support for Additional Privacy Constraints.** The modular architecture can be extended to support other privacy constraints, including Chinese Wall policies, need-to-know principles, and RBAC constraints.

- **Performance Optimization.** The state space of the augmented Petri nets can grow significantly with the number of participants, knowledge items, and duties. Future work could explore optimization techniques to reduce the state space and improve verification performance.
- **Industrial Validation.** The framework should be validated on larger, more realistic process models from industrial settings to confirm its scalability and practical applicability.

8.7 Final Remarks

This thesis has demonstrated that privacy constraints such as SoD, DoK, and BoD can be effectively verified in BPMN choreography diagrams by extending existing Petri net-based verification tools. The key to success is correct net augmentation, which accurately models the knowledge and duty states of participants. Once the augmentation is correct, the specification generation is straightforward, and verification can be performed automatically using standard model-checking techniques.

The resulting framework bridges the gap between privacy-aware modeling and formal verification, enabling practical analysis of privacy constraints in cross-organizational collaborations. By making privacy verification accessible within the same framework used for behavioral verification, this thesis contributes to the broader goal of building trustworthy and privacy-preserving distributed systems.

References

[1] V. Atluri and W. K. Huang. Enforcing mandatory access control in workflow systems. *ACM Transactions on Information and System Security*, 2(1):62–91, 1999.

[2] A. Awad, G. Decker, and M. Weske. Efficient compliance checking using BPMN-Q and temporal logic. *Information Systems*, 36(6):956–978, 2011.

[3] Sara Belluccini et al. Model-based verification of data protection mechanisms in collaborative business processes. *Information Systems*, 2025. In press.

[4] A. Bertolino, S. Gnesi, and A. Martens. Privacy-aware analysis of business process models. *Journal of Systems and Software*, 186:111–128, 2022.

[5] Edmund M. Clarke, Orna Grumberg, and Doron A. Peled. *Model Checking*. MIT Press, 1999.

[6] Marlon Dumas, Marcello La Rosa, Jan Mendling, and Hajo A. Reijers. *Fundamentals of Business Process Management*. Springer, 2nd edition, 2018.

[7] H. Groefsema. *Business Process Variability: A Study into Process Management and Verification*. PhD thesis, University of Groningen, 2016.

[8] H. Groefsema, N. R. T. P. van Beest, and A. Armas-Cervantes. Automated compliance verification of business processes in apromore. In *Proceedings of the BPM Demo Track*, pages 1–5, 2017.

[9] Sebastian Groll, Ludwig Fuchs, and Günther Pernul. Separation of duty in information security. *ACM Computing Surveys*, 57(7):1–35, 2025.

[10] K. Jensen and L. M. Kristensen. *Coloured Petri Nets: Modelling and Validation of Concurrent Systems*. Springer, 2009.

[11] K. Knorr. Dynamic access control through petri net workflows. *Proceedings of the 16th Annual Computer Security Applications Conference*, pages 159–167, 2000.

[12] M. Montali, M. Pesic, and W. M. P. van der Aalst. Declarative process modeling and verification. *Information Systems*, 38(7):1010–1032, 2013.

[13] Tadao Murata. Petri nets: Properties, analysis and applications. *Proceedings of the IEEE*, 77(4):541–580, 1989.

[14] NuSMV Team. *NuSMV: A New Symbolic Model Checker*, 2025. Version 2.6.0.

[15] Object Management Group. Business process model and notation (bpmn) version 2.0, 2011. Accessed: 2026-06-20.

[16] J. Pullonen, M. Leppänen, and T. Soininen. Privacy-enhanced bpmn for separation of duties and division of knowledge. *International Journal of Information Security and Privacy*, 6(3):45–62, 2012.

[17] Wolfgang Reisig. *Petri Nets: An Introduction*, volume 4 of *EATCS Monographs on Theoretical Computer Science*. Springer, 1985.

[18] K. Schmidt. LoLA: A low level analyser. *Lecture Notes in Computer Science*, 1825:465–468, 2000.

[19] Bruce Silver. *BPMN Method and Style*. Cody-Cassidy Press, 2nd edition, 2011.

[20] Wil M. P. van der Aalst. *Process Mining: Data Science in Action*. Springer, 2016.